

Control available in SMC

Marko Guberina

September 25, 2025

0.1 Math notation

Just to avoid confusion, let's explicitly define the notation used in the rest of the text.

- $q \in \mathbb{R}^{nq}$ - joint configuration vector
- $v \in \mathbb{R}^{nv}$ - joint velocity vector
- $a \in \mathbb{R}^{nv}$ - joint acceleration vector
- $\tau \in \mathbb{R}^{nv}$ - joint torque vector
- $u \in \mathbb{R}^{nu}$ - control input (almost always the velocity command vector $v_{cmd} \in \mathbb{R}^{nv}$)
- $T_a^b \in SE(3)$ - element of special Euclidean group, representing the frame $\{b\}$ as seen in frame $\{a\}$. Most often the end-effector frame as seen in the world frame T_w^e . It's its own $SE(3)$ object in code with functions like `act()` which apply the transformation to a different object. Mathematically, unless explicitly stated otherwise, it's should be considered as a homogeneous transformation matrix

$$T_a^b = \begin{bmatrix} R_a^b \in \mathbb{R}^{3 \times 3} & p_a^b \in \mathbb{R}^3 \\ 0 \in \mathbb{R}^{1 \times 3} & 1 \end{bmatrix} \quad (1)$$

- $J_a^b(q)$ - Jacobian of the $T_a^b(q)$ map which depends on the joint configuration q . Most often $J_w^e(q)$ and abbreviated to just J , but should be obvious what it is within the context. **VERY IMPORTANT NOTE** by default, all Jacobians are **body Jacobians**, meaning they map to velocities in the end-effector frame $\{e\}$, **not** in the world frame $\{w\}$. This is because Pinocchio does kinematics and dynamics with the Lie group approach, and it's more computationally efficient to compute quantities in the body frame. Space Jacobians can be computed easily by passing an additional argument to the `pinocchio::getFrameJacobian()` function, but this is not done in the SMC framework *at all*.
- e - generic error vector

- twist V - combined vector of linear v and rotational velocity ω , $V = \begin{bmatrix} v & \omega \end{bmatrix}^T$
- wrench F - combined vector of linear force f and torque τ , $F = \begin{bmatrix} f & \tau \end{bmatrix}^T$

1 Joint-space control

There are just 2 controllers. A P-controller with feedback on positions for point to point motion and a P-controller with feedback on positions and feedforward velocity for trajectory following. Since they get the job done for the rare cases they are needed in, they haven't been developed further. There is a PI controller for point to point motion as well, but it's never been used for anything.

moveJP

1. $e = q_{\text{desired}} \ominus q$
2. $v_{\text{cmd}} = K \cdot \|e\| \cdot \frac{1}{\text{freq}}$

Note that the controller below is implemented in a very ugly way in code.

followKinematicJointTrajP

1. $q_{\text{desired}} = \text{interpolate}(q_{\text{reference}}(t))$
2. $q_{\text{error}} = q_{\text{desired}} - q$
3. $v_{\text{desired}} = \text{interpolate}(v_{\text{reference}}(t))$
4. $v_{\text{cmd}} = v_{\text{desired}} + K_p \cdot q_{\text{error}}$

2 Cartesian control

There is a generic point-to-point a generic trajectory following `ControlLoopTemplate`. The point-to-point template computes the error vector as

$$e = \log_6 \left((T_w^e)^{-1} \cdot T_w^{\text{goal}} \right) \quad (2)$$

which means e is a twist (6D velocity), i.e. $e \in se(3)$. It then calls on an `ik_solver` which is specified with an argument. The available inverse kinematics solvers are listed below they all take the error twist and the Jacobian as arguments, and return their joint velocity solution.

The only thing that the path-following does differently is how the

2.1 Point to point cartesian control

Point to point cartesian control

1. compute $SE(3)$ error $E = \log_6 \left((T_w^e)^{-1} \cdot T_w^{\text{goal}} \right) = T_e^{\text{goal}}$, i.e. the transformation from end-effector frame to goal frame
2. map to $se(3)$ via the log map: $V_{\text{error}} = \log_6(T_e^{\text{goal}})$.
3. compute joint velocity v such that the end-effector moves with twist V_{error} with an ik_solver with $v_{\text{cmd}} = \text{ik_solver}(J, v_{\text{error}})$

Short note on the log operation The twist V_{error} corresponds to the velocity such that moving from frame $\{e\}$, with twist V_e , for one unit of time, leads to the $\{g\}$ frame. Said another way, if $\{V_e\}$ is the body velocity of the end-effector frame for one unit of time, it gets to the goal frame. The reason this works has to do with Lie algebra. Detailed overview can be found in ex. Frank C. Park, Kevin Lynch - Modern Robotics or Murray, R.M., Li, Z. and Sastry, S.S., 2017. A mathematical introduction to robotic manipulation. CRC press, or on [this excellent blog post](#).

2.2 Inverse kinematics solvers

There are of course the classics:

- Jacobian pseudoinverse: $v = J^\dagger V_e = J^T(JJ^T + \lambda I)^{-1} V_e$
- Jacobian transpose: $v = J^T V_e$

For larger movement, these methods can result in wonky trajectories as the error twist V_e is large, resulting in large joint velocity commands v . Those in turn get saturated, leading to the end-effector not going in a straight line, but something else. This could be solved by scaling the error twist V_e to a smaller number, but this is not an ideal solution because the maximum end-effector twist in a particular direction depends on the configuration (which could be taken into account manually, but there are better methods out there).

QP formulation The inverse kinematics problem can be posed as an optimization problem:

$$\min_v \frac{1}{2} \|J\Delta q + wV\| \quad (3)$$

$$\text{s.t. } v_{\min} \leq v \leq v_{\max} \quad (4)$$

The joint displacement Δq can be regarded as joint velocity v if the integration step is small enough (it is, i.e. we make sure that we have high enough control

frequency). From now on we write v . If there were no additional objectives, and no any constraints whatsoever, the linear least-squares solution to this problem would be the Jacobian pseudoinverse method.

This can be transcribed as a generic quadratic program (QP), which looks like

$$\min_x \frac{1}{2} x^T P x + q^T x \quad (5)$$

$$\text{s.t. } Gx \leq h \quad (6)$$

$$Ax = b \quad (7)$$

In particular, the transcription goes as follows:

$$\|Jv - KV\|^2 = (Jv - KV)^T (Jv - KV) \quad (8)$$

$$= v^T J^T J v - KV^T J v - Kv J^T V + \underbrace{K^2 V^T V}_{\text{does not depend on } v} \quad (9)$$

$$\implies v^T \underbrace{(J^T J)}_P v - \underbrace{2KV^T J v}_r \quad (10)$$

Note that the same Tikhonov regularization (also known as Levenberg-Marquardt damping in this context) should added to the P as it was in the pseudoinverse, i.e. $P = J^T J + \lambda I$. Crucially, we can add additional tasks and constraints to the QP formulation, as long as the costs are quadratic, and the constraints are linear. Since we are solving this problem at each control time step, this is actually quite powerful as we can use linearised nonlinear costs/constraints and they will work fine.

In particular, let a cost be denoted as $e(q)$, and it's Jacobian by $J(q)$. This will again result in a cost of form $\|J_i v - K_i e_i\|^2$. You then work out the same steps as above for the frame placement cost, getting (P_i, r_i) . Then you can simply put $P = \sum_i P_i, r = \sum_i r_i$, which is still quadratic. This emulates behaviour of HQP.

Finding an appropriate secondary task, and doing that computation is a part of the TODO. Since the biggest issue on YuMi is the elbows hitting the torso, going for $e = q_{\text{comfy_configuration}} - q$ is probably the way to go. The alternatives are manipulability maximization or self-collision avoidance. Manipulability maximization could also be added as a ternary goal, where the manipulability is defined for the whole of the robot. Looking at [pink](#) and [it's documentation](#) is a good place to start. **Note** that pink isn't integrated into SMC simply because it was annoying to match it's structure to SMC's, but it can be done. A starting point for that approach would be to look into `examples/cartesian_space/yumi_self_collision.py`.

Solution approach We need the QP formulation of inverse kinematics, (or something which allows for additional objectives and constraints), because the pseudoinverse solver causes the configuration of the robot goes to shit (breaks the robot) during Cartesian path-following (and thus cart-pulling as well).

One way to implement the QP formulation is to shove the whole of pink setup into an `ik_solver` construction. Then it can be applied to any `controlLoop` without any additional code. That requires creating a big struct for all pink's variables, which is then partialized to comply with `ik_solver`'s API. The API could also be changed, just note that you then need to refactor this throughout the codebase.

The way adding tasks is handled in pink is great, and there is no point in rewriting that. We want this because it would make it super easy to swap different objectives, i.e. we wouldn't need to a new ik solver for every thing to try (or write the logic equivalent to pink to make one of the customizable).

Alternatively, we can manually construct the QP, and also partial in whatever additional variables we need. The drawback is that this will be more verbose and annoying, but the upside is that the math of the QP will be completely transparent.

The code for all inverse kinematics solvers is in `python/smc/control/cartesian_space/ik_solvers.py`.

Available QP solvers:

- solvers supported by [quadprog](#) (we use what the default one is, but you can install and select any one supported by quadprog)
- [proxsuite](#) - cool because it has warmstarting (among other things)

Secondary goals

- posture reference tasks with $e(q) = q_{\text{comfy}} - q \rightarrow \text{DONE}$
ik_solver is called `QPWithPosture`. It does not take mobile poses positions into account of course.
- velocity limit constraints $\rightarrow \text{DONE}$
- TODO: joint limit constraints
- TODO IF ABOVE ISN'T ENOUGH FOR GOOD POSTURE: put in the derivative of manipulability w.r.t. q .
- TODO IF SELF-COLLISIONS ARE COMMON: adding collision avoidance via control barrier functions. Regarding what to avoid, there are 2 options:
 1. make spheres around elbows and end-effectors. Use `BodySphericalBarrier` from pink for this. Inputs are frames that can't be avoided and the radius of the sphere around said frames. Look at `pink/examples/barriers/yumi_self_collision.py` for details. There's an SMC-ed version of this in `examples/cartesian_control/yumi_self_collision.py`
 2. make ellipses around every link. Use `SelfCollisionBarrier` from pink for this. Look at `pink/examples/barriers/kukas_self_collision.py` for details.

How to run The main file for SingleArmInterface point to point Cartesian control is `example/cartesian_space/clik_point_to_point.py`, and the one for DualArmInterface is `example/cartesian_space/dual_arm_clik.py`. The relevant arguments can be found simple by adding `-help` when running the example, but the most important ones is `-ik-solver [solver]` to select a solver.

2.3 Path following Cartesian control

It's sad how stupidly Cartesian path following is implemented.

It works on $SE(3)$ paths, i.e. there are no pre-determined timing laws. The controller assumes that the first point of the path coincides with the current pose of the thing we're planning for. The responsibility to ensure this is on the path-planner (the current pose is accessible to it). While one might argue that there should be a handling routine for when this isn't the case, because the path planner is slower than the controller, we simply don't have this problem at the moment, so we haven't solved it.

The path-following controller computes the positional error twist the same way as the point-to-point to point version, from the end-effector to the second point of the path. Does the same between the second and third point of the path, and treats that as a reference velocity. This is obviously stupid because the quality of it depends on spacing of the path points, and there are circumstances where this causes performance issues. [In Marko's opinion] It is the responsibility of the controller to smooth out the given path (from the web programming mantra "be loose with what you accept, be rigid in what you provide" (yes this is in opposition to assuming the first path point coincides with the end-effector, but again, that wasn't a problem (yet))).

[ABHISHEK TODO]: This is to be fixed with path interpolation. The algorithm should then act as follows:

Cartesian path following controller

1. (optional) find the current path point (where you are on the path)
2. determine how many points in advance should be interpolated (as few as possible)
3. interpolate those points to create a smooth reference (poly, Bezier or w/e)
4. (possibly necessary) low-pass filter update this with the previous path in the case of following from a path planner, to have smoother motion if/when the plan changes by a lot
5. impose some timing law on the path as then makes positional control makes sense. The timing law choice should be connected to the heuristic on number/length of interpolated path points .
6. compute desired velocity (most likely feedback position error, feedforward velocity, but this probably depends on how the interpolation is done??? idk)
7. apply an inverse kinematics solver to find joint velocities to achieve desired velocity

More details on this are given below.

The existing code can be found in

```
python/smc/control/cartesian_space/cartesian_space_trajectory_following.py .
```

[ABHISHEK TODO]: path interpolation This is the key thing to do to make sure Cartesian path following works. The first thing to do is determine the number of points to be interpolated between. There is no benefit in interpolating the whole path as we're not following the whole path. We want a simple curve to make the computation quick, as it needs to happen in real-time in conjunction with the real-time path planner (anything reasonable should be fine). **Note** that linear interpolation (without splines) isn't good enough as it results in jagged motion on the "edges".

One option is to form a Bezier curve, or a polynomial interpolation on the first few points and call it a day. A smarter version would be to do the same, but with least-squares fit on a larger number of points, the amount of which is determined by some heuristic like their arclength being at least something.

Deciding on the exact strategy is a part of the TODO.

Short note on 2D to 6D reference Often we get a 2D path to follow, which is of course underdefined. This is most common when drawing a path for the purposes of testing a path-following controller. The path is made to be 3D by specifying a height h (this is an argument), giving the translation

$p = [x_i \ y_i \ h]$ Of course, we could impose an arbitrary orientation along the path, and interpolate the $SO(3)$ rotations to make it smooth. However, this isn't implemented as it (so far) isn't useful for cart-pulling nor navigation.

Instead, a Frenet frame is constructed on the linear path to give it orientation. First, we make the x-axis be the tangent of the path. The angle of the x-axis of the frame as seen in the x-y plane of the world frame $\{w\}$ is determined with

$$\theta[i] = \text{atan2} \left(\frac{\phi_y[i+1] - \phi_y[i]}{\phi_x[i+1] - \phi_x[i]} \right) \quad (11)$$

and we turn this into a rotation matrix with $R_{\theta_i} \text{rpyToMatrix}(0, 0, \theta_i)$ Next, we specify the rotation about the x-axis (this is an argument), and get $R_x = \text{rpyToMatrix}(\text{rot_x}, 0, 0)$. In cart-pulling, we want the z axis to be perpendicular to the The y-axis completes the right-handed frame, meaning $R = R_{\theta} \cdot R_x$. Finally, the resulting $SE(3)$ path is then

$$P_i = \begin{bmatrix} R_i & p_i \\ 0 & 1 \end{bmatrix} \quad (12)$$

How to run The main file for SingleArmInterface path following Cartesian control is `example/cartesian_space/clik_path_following.py`, and the one for DualArmInterface is `example/cartesian_space/dual_arm_clik_path_following.py`.

The relevant arguments can be found simple by adding `-help` when running the example, but the most important ones is `-ik-solver [solver]` to select a solver. Additionally, there is a `-planner` option (on by default). This will spawn the real-time path generation in a new process, and create a predefined map.

3 Optimal control

We use the Box-FDDP solver [reference] from the Crocoddyl optimal control suite [reference]. It's a DDP-style trajectory optimizing method. It works on the dynamics level. Since we only do velocity control, we use the velocity obtained from the forward rollout as the actual control input.

Crocoddyl's main benefits are that it is (very) fast, and that it uses Pinocchio for it's dynamics computations (same as the rest of the codebase). It's main drawback is the verbosity of the API, and the difficulty of defining novel costs or constraints. This is because Crocoddyl is written in C++, and just pybind's to expose it's API in Python. Meaning, if something is missing, it has to be written in C++, and then exported to Python. While this isn't prohibitively difficult, it entails compiling the library (and likely it's dependencies) and requires a working knowledge of C++, Pybind and the library design. In other words, this should be pursued only if there is a good reason to do so as the required implementation time will detract from other tasks.

A generic optimal control problem looks like

$$X^*, U^* = \operatorname{argmin}_{X, U} \sum_{k=1}^{N-1} \int_{t_k}^{t_{k+1}} l(x, u) dt + L(x(N)) \quad (13)$$

$$\text{such that } \dot{x} = f(x, u) \quad (14)$$

$$x \in \mathcal{X}, u \in \mathcal{U} \quad (15)$$

$$(16)$$

A common choice for $\Delta t = t_{k+1} - t_k$ is $\Delta t = 0.01$.

Since we are working exclusively with trajectory optimization, we can make use of the additional structure of the solver. Box-FDDP is a direct multiple-shooting trajectory optimization method. Multiple-shooting means that states x_i are also decision variables. Trajectory optimization means that we work with a candidate solution, or a solution guess which is iteratively improved. One iteration consists of:

1. A forward simulation of the dynamics with the current control input candidate, which includes the costs. This is referred to as the forward pass.
2. Linearising the dynamics and the costs around every point the obtained trajectory.
3. Updating the candidate solution by performing a line search in the direction of most decreasing cost. This is referred to as the backward pass.

The software structure of this is a list of N `actionModel`s, each of which contains a dynamics model and a cost model. Since the cost is in the end a scalar, this means that these action models do not need to be the same — the dynamics, cost, linearization and line search update computations are performed at each action model (also referred to as knot or node) individually. We use this fact in the path-following formulation, more on that later. Most importantly, we add a much larger goal cost on the last node, called the terminal node.

Depending on the task, we create a different objective function. Before doing that, we introduce the specific generic parts we always have, and a short note on how to go from optimal control for planning to real-time MPC.

3.1 Turning OCPs into MPCs

Any OCP described here is turned into an MPC by:

- **Shortening the time horizon N , capping the maximum number of solver iterations $n_{\text{solver_iter}}$, and reducing control frequency f_{ctrl}** until satisfactory performance is reached. From limited qualitative testing, the most common numbers are $N = 30$, $n_{\text{solver_iter}} = 10$, and $f_{\text{ctrl}} = 200$. In code this is done simply by passing arguments.

- **Warmstarting** with the previous solution. The previous solution is offset by the amount of time passed. TODO: make sure it is, make sure the ctrl_freq amount of time, not Δt time. If the second is the case, you probably want to linearly interpolate to offset correctly - also make sure you're time-testing whether this is actually helpful (if it doesn't shave off an iteration off of the solver, it isn't)
- don't have zero velocity in the terminal node

In the case that the decreased control frequency causes problems, some kind of low-pass or similar filtering can be applied to the incoming control signals, smoothening them out, but this hasn't been done.

3.2 Generic OCP components

Every OCP we construct has these components.

Decision variables, dynamics constraint In our case, the state is a vector of joint positions and velocities:

$$x = [q \quad v]^T. \quad (17)$$

The control inputs can be either acceleration or torques, depending on which we formulate either the inverse or the forward dynamics problem respectively, and solve for the other quantity. We used inverse dynamics. In any case, because the states are also decision variables, a state-input pair must follow the system dynamics into the next that. Thus to put in the discrete version of $\dot{x} = f(x, u)$, we can write

$$\text{dynamics}(\delta x_{k+1}, \delta x_k, \delta u_k) = 0 \quad (18)$$

$$\text{dynamics}(\delta \dot{x}, \delta x, \delta u) = \delta \dot{x} - (f_x \delta x + f_u \delta u) \quad (19)$$

State constraints Since we use Box-FDDP, we can only have box constraints on torque inputs, i.e. $\tau_{\min} \leq \tau \leq \tau_{\max}$. In the case of an underactuated joint, we set the torque limit on it to zero.

To put constraints on the state, namely on arms' joint limits and on all joints' velocities, we put them into a quadratic barrier function which we put into the objective function. In particular (TODO make sure this is correct):

$$r_{lb} = x - x_{\min} \quad (20)$$

$$r_{ub} = x - x_{\max} \quad (21)$$

$$B(x) = \begin{cases} \frac{1}{2}(\|r_{lb}\|^2 + \|r_{ub}\|^2) & \text{if } lb \leq x \leq ub \\ \text{max float} & \text{otherwise} \end{cases} \quad (22)$$

Regulation costs (regularization) Finally, we add regulation costs on both the state $c_1 x^T x$, and control inputs $c_2 u^T u$.

3.3 Point to point

In case there is a fixed end-goal to reach, we use the $se(3)$ error norm as the task-specific function to put into the objective function.

Single arm end-effector reference TODO In the particular case of the goal being defined for the end-effector, whose pose is denoted as T_w^e reaching a goal denoted as T_w^{goal} :

$$l_e(x) = \left\| \log \left((T_w^e)^{-1} \right) T_w^{\text{goal}} \right\|_2 \quad (23)$$

Here we of course use only joint values q , not the full state x , because the end-effector pose is only a function of q .

We put this cost function at every knot. This method works fine.

How to run The main is in `examples/optimal_control/point_to_point/croco_ee_reference_p2p_mpc`

Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Base and single arm end-effector reference In addition to the end-effector distance to goal cost, we add the same one for the base. Despite the base being an $SE(2)$ joint, we use an $SE(3)$ error as well. This is simply because this is easier to do within the Crocoddyl framework, and it of course doesn't change anything. Thus the objective is given by:

$$l_e(x) = \left\| \log \left((T_w^e)^{-1} \right) T_w^{\text{ee-goal}} \right\|_2 + \left\| \log \left((T_w^b)^{-1} \right) T_w^{\text{base-goal}} \right\|_2 \quad (24)$$

Alternatively, we can define the base cost to be defined only through position (i.e. have no orientation cost). The objective is then

$$l_e(x) = \left\| \log \left((T_w^e)^{-1} \right) T_w^{\text{ee-goal}} \right\|_2 + \|p_{\text{base-goal}} - p_{\text{base}}\|_2 \quad (25)$$

We put this cost function at every knot. This method works fine for the most part, but has trouble converging right at the end. It looks as if it continues to actuate the base to move the end-effector, despite the base being already at the goal. This is particularly the case with a position-only cost in the base. A simple hack would be to let a Cartesian control end the control loop after the overall error is small enough.

How to run The main is in `examples/optimal_control/point_to_point/croco_base_and_ee_reference`

Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Dual arm end-effector reference The objective is simply the sum of left end-effector goal cost and right end-effector goal cost:

$$l_e(x) = \left\| \log \left((T_w^{e_l})^{-1} \right) T_w^{\text{l-ee-goal}} \right\|_2 + \left\| \log \left((T_w^{e_r})^{-1} \right) T_w^{\text{r-ee-goal}} \right\|_2 \quad (26)$$

Discussion It doesn't work because it can't converge — it gets pretty close, and then oscillates.

How to run The main is in `examples/optimal_control/point_to_point/croco_dual_ee_reference_p2p`

Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Base and dual arm end-effector reference As expected, this is just a combination of the base and each end-effector costs:

$$l_e(x) = \left\| \log \left((T_w^{e_l})^{-1} \right) T_w^{l_ee_goal} \right\|_2 + \left\| \log \left((T_w^{e_r})^{-1} \right) T_w^{r_ee_goal} \right\|_2 + \|p_{base_goal} - p_{base}\|_2 \quad (27)$$

How to run The main is in `examples/optimal_control/point_to_point/croco_base_and_dual_ee_refe`

Remember to set `-n-knots` to something small like 30, and `-max-solver-iter` to something like 10 so that this runs in reasonable time!

Discussion Very bad performance. Not only does it not converge, but the arms move in a very strange way.

4 Trajectory following

The optimal control formulation allows us to optimize the time-law over the path. In other words, the total time for the path, or a velocity over it does not need to be imposed, but could instead be solved for within the optimal control problem as an additional decision variable. Let the prescribed path be parametrized by $s \in [0, 1]$. One simple approach would be to minimize the time taken to traverse the path by minimizing the time T which can be a function of $s \mapsto Ts \in [0, T]$. In fact this could be the only cost, while following the path can be considered as a constraint.

This has not been done in SMC due to development limited time. Instead, we assume a uniform velocity over the path to obtain a Cartesian trajectory. We then sample from this trajectory and set those points to be distance costs at corresponding action models. In other words, the OCP is identical to the point to point OCP, but instead of the error at every action model being towards the final goal, it's to the corresponding point on the trajectory. Despite this limitation, the path following versions work about as well as the point-to-point ones.

5 [MARKO TODO] Navigation (mobile base control)

[MARKO TODO] **Following a path planner** Always expect the initial point to be in the current position of the robot (our planner is fast)

[MARKO TODO] **Following a prescribed path** TODO: 1) introduce the function to get you to the closest point. 2) explain the need to smoothly interpolate between points (sometimes the given points are very sparse).

6 Dynamic motion primitives

TODO